The Modal Axiom-Schema Architecture

`SchemaUnion`, `ModalSchemaTag`, and the `unionSound` hinge

CSLib — Internal Document

— July 19, 2026 —

Internal document. This report renders the architecture note `docs/modal-axiom-schema-architecture.md` as a typeset deliverable. It is a derived, differently-purposed artifact – the Markdown source remains the primary reference and is unchanged by this document. Definitions, lemma signatures, and axiom schemas are transcribed from the landed, CI-green Lean source cited in each section's anchors, not from memory.

1 Overview

CSLib's modal-logic layer formalizes 15 classical normal modal systems — K, T, D, B (a.k.a.
KB), K4, K5, K45, S4, S5, TB, KB5, D4, D5, D45, DB — each as a Hilbert-style proof system over a shared propositional/modal `Proposition` language. Each system's axiom set was historically a bespoke, hand-written `inductive` with its own 13–18 constructors, byte-identical to its siblings on a shared propositional/K/and-or/diamond-duality core and differing only in which modal-strength schemata (T, D, B, 4, 5) it admits. This document describes the compositional architecture that replaced those hand-written inductives: **15 systems, unified under 14 renamed `<Sys>Axiom` abbreviations plus S5′s pre-existing `ModalAxiom`**, all expressed as one-line `abbrev`s over a single combinator, `SchemaUnion`, and a single 18-tag alphabet, `ModalSchemaTag`.

The architecture rests on two libraries, two halves of one abstraction:

- **`SchemaUnion.lean`** (+ `SchemaTags.lean`) — the *syntactic* side: the tag alphabet, the existential "schema = set of instances" meaning function, the `SchemaUnion` combinator, and the elimination lemmas that make the modal cube's $\subseteq$-order a `decide`-able computation on `Finset ModalSchemaTag` values.
- **`FrameCorrespondence.lean`** (+ `SchemaSoundness.lean`) — the *semantic* side: the five frame-condition-to-validity correspondence lemmas for the modal-strength differentiators, and the master soundness lemma, `unionSound`, hinging the two sides together.

Two payoffs fall out directly: **subsumption** collapses from 24 hand-written per-edge lemmas to one generic `SchemaUnion.subsumption` application discharged by `decide`-able `Finset.subset` facts (Section 2), and **soundness** collapses from 15 per-system case-splits to one `unionSound` application consuming an 18-entry `FrameValidatesTag` table plus the five frame-correspondence lemmas (Section 3, "`unionSound`: The Syntax/Semantics Hinge").

2 The Schema-Tag Alphabet: `ModalSchemaTag`, `.Holds`, and `SchemaUnion`

Durable anchor: `Cslib/Logics/Modal/ProofSystem/SchemaUnion.lean`.

Every one of the 15 classical systems' axiom set is built from the same finite vocabulary of axiom *schemata*. `ModalSchemaTag` names that vocabulary as an 18-constructor alphabet, grouping into five roles (4 + 1 + 5 + 6 + 2 = 18):

Role	Tag	Schema
Propositional core	`implyK`	$\phi \to (\psi \to \phi)$
	`implyS`	$(\phi \to (\psi \to \chi)) \to ((\phi \to \psi) \to (\phi \to \chi))$
	`efq`	$\bot \to \phi$
	`peirce`	$((\phi \to \psi) \to \phi) \to \phi$
K-distribution	`modalK`	$\Box(\phi \to \psi) \to (\Box\phi \to \Box\psi)$
Modal-strength differentiators	`modalT`	$\Box\phi \to \phi$
	`modalD`	$\Box\phi \to \Diamond\phi$
	`modalB`	$\phi \to \Box\Diamond\phi$
	`modalFour`	$\Box\phi \to \Box\Box\phi$
	`modalFive`	$\Diamond\phi \to \Box\Diamond\phi$
And/or	`andI`	$\phi \to (\psi \to (\phi \wedge \psi))$
	`andE1`	$(\phi \wedge \psi) \to \phi$
	`andE2`	$(\phi \wedge \psi) \to \psi$
	`orI1`	$\phi \to (\phi \vee \psi)$
	`orI2`	$\psi \to (\phi \vee \psi)$
	`orE`	$(\phi \to \chi) \to ((\psi \to \chi) \to ((\phi \vee \psi) \to \chi))$
Diamond duality	`diaDualityFwd`	$\Diamond\phi \to \neg\Box\neg\phi$
	`diaDualityBack`	$\neg\Box\neg\phi \to \Diamond\phi$

Table 1: The 18 `ModalSchemaTag` constructors, grouped by role. Schemas are transcribed verbatim from each constructor's doc comment in `SchemaUnion.lean`.

A tag by itself is just a label; `ModalSchemaTag.Holds` gives each tag its formula-level meaning as an existential over the schema's metavariables — the "schema = set of its instances" encoding used throughout:

```
def ModalSchemaTag.Holds : ModalSchemaTag → Proposition
Atom → Prop
  | .implyK, χ => ∃ φ ψ : Proposition Atom, χ = φ.imp
(ψ.imp φ)
  | .modalT, χ => ∃ φ : Proposition Atom, χ =
(Proposition.box φ).imp φ
  | .modalB, χ => ∃ φ : Proposition Atom, χ = Axioms.AxiomB
φ
  -- … 15 further clauses, one per remaining tag
```

Every clause's proposition shape matches, byte-for-byte, the corresponding constructor of the pre-existing per-system axiom inductives, which is what makes it possible to replace those inductives with a single combinator, parametrized by a chosen `Finset` of tags:

```
def SchemaUnion (S : Finset ModalSchemaTag) : Proposition
Atom → Prop :=
  fun χ => ∃ t ∈ S, t.Holds χ
```

`SchemaUnion S χ` holds iff `χ` is an instance of some tag in `S`. Because the alphabet is finite and carries `DecidableEq`, choosing `S` to be a concrete `Finset` literal turns each system's axiom predicate into a one-line declaration. Fourteen of the fifteen systems now read `abbrev <Sys>Axiom := SchemaUnion sysTags` in `ProofSystem/Instances/{K,T,D,B,K4,K5,K45,S4,TB,KB5,D4,D5,D45,DB}.lean`. S5 is not a structural exception: `Metalogic/DerivationTree.lean` line 69 reads exactly

```
abbrev ModalAxiom : Proposition Atom → Prop := SchemaUnion
s5Tags
```

so `ModalAxiom` is the 15th system folded into the same one-line-abbrev-over-`SchemaUnion` pattern as the other 14, its name and public API preserved by redefinition-in-place.

Alongside the combinator, three `@[simp]` elimination lemmas give the ergonomic half of the design: a concrete `SchemaUnion sysTags φ` obligation rewrites via `simp` into the named disjunction of its tags' `.Holds` clauses, rather than requiring raw `fin_cases` destructuring at every call site:

```
theorem SchemaUnion.empty_iff {φ : Proposition Atom} :
    SchemaUnion (∅ : Finset ModalSchemaTag) φ ↔ False

theorem SchemaUnion.insert_iff {t : ModalSchemaTag} {S :
Finset ModalSchemaTag}
    {φ : Proposition Atom} :
    SchemaUnion (insert t S) φ ↔ t.Holds φ ∨ SchemaUnion S
φ

theorem SchemaUnion.union_iff {Sa Sb : Finset
ModalSchemaTag} {φ : Proposition Atom} :
    SchemaUnion (Sa ∪ Sb) φ ↔ SchemaUnion Sa φ ∨
SchemaUnion Sb φ
```

Durable anchors: `SchemaUnion.lean` (`ModalSchemaTag`, `.Holds`, `SchemaUnion`, the three `@[simp]` lemmas); `Metalogic/DerivationTree.lean` (`ModalAxiom`, line 69); `ProofSystem/Instances/{K,…,S5}.lean` (the 15 per-system instance-registration files).

3 Subsumption as `Finset.subset`: The Syntactic Modal Cube

Durable anchors: `SchemaUnion.subsumption` (`SchemaUnion.lean`), `SchemaTags.lean` (the 15 per-system tag sets), `Metalogic/InterSystem/AxiomSubsumption.lean` (the 24 direct-edge lemmas).

The modal cube — the familiar inclusion order K ⊆ T ⊆ S4 ⊆ S5, and its 11 further systems and their edges — used to be witnessed by 24 hand-written, per-edge subsumption lemmas, each proved by a bespoke case analysis over the source system's axiom inductive. `SchemaUnion` replaces all 24 with one generic lemma:

```
theorem SchemaUnion.subsumption {Sa Sb : Finset
ModalSchemaTag} (hsub : Sa ⊆ Sb)
    {φ : Proposition Atom} (h : SchemaUnion Sa φ) :
SchemaUnion Sb φ
```

Enlarging the tag set only weakens the resulting predicate — the proof is three lines (`obtain ⟨t, ht, hφ⟩ := h; exact ⟨t, hsub ht, hφ⟩`). Every `XAxiom_implies_YAxiom` lemma across the cube is now `SchemaUnion.subsumption (by decide) h`: a `decide`-able `Finset.subset` fact, discharged automatically because `ModalSchemaTag` is a finite `DecidableEq` type and every system's tag set is a concrete `Finset` literal built from `insert`/`kCore`. This is the framing point of the whole design: **the modal cube's ⊆-order IS the ⊆-order on tag sets** — subsumption is no longer a proved fact about 15 independent inductives, it is a computation on 15 `Finset` values.

3.1 The 15 tag sets

Each of the 15 per-system tag sets (`SchemaTags.lean`) is the shared 13-tag core, `kCore`, unioned with a subset of the 5 modal-strength differentiator tags:

```
def kCore : Finset ModalSchemaTag :=  -- 13 tags: implyK,
implyS, efq, peirce,
  …                                    -- modalK, andI,
andE1, andE2, orI1,
                                        -- orI2, orE,
diaDualityFwd, diaDualityBack
```

System	Tag set
K	`kCore`
T	`kCore` ∪ {modalT}

System	Tag set
D	`kCore` ∪ {modalD}
B (KB)	`kCore` ∪ {modalB}
K4	`kCore` ∪ {modalFour}
K5	`kCore` ∪ {modalFive}
K45	`kCore` ∪ {modalFour, modalFive}
S4	`kCore` ∪ {modalT, modalFour}
S5	`kCore` ∪ {modalT, modalFour, modalB}
TB	`kCore` ∪ {modalT, modalB}
KB5	`kCore` ∪ {modalB, modalFive}
D4	`kCore` ∪ {modalD, modalFour}
D5	`kCore` ∪ {modalD, modalFive}
D45	`kCore` ∪ {modalD, modalFour, modalFive}
DB	`kCore` ∪ {modalD, modalB}

Table 2: The 15 per-system tag sets, each `kCore` unioned with a subset of the 5 differentiator tags (`SchemaTags.lean`).

3.2 The syntactic modal cube

The 24 direct edges in `AxiomSubsumption.lean` — each now a one-line `SchemaUnion.subsumption (by decide) h` proof, where previously each was a hand-written per-edge `match`/`cases` lemma — form a layered DAG on the 15 tag sets, ordered by number of differentiators:

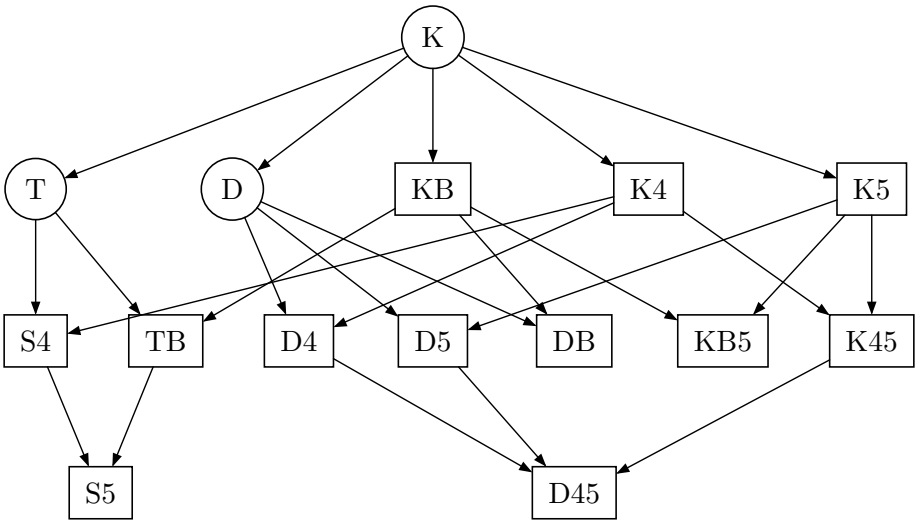

Figure 1: The 24-edge syntactic modal cube (`AxiomSubsumption.lean`), layered by differentiator count. Every edge is a `decide`-able

`Finset.subset` fact between the two systems' tag sets (Table 2). Notably absent: KB5 → S5 (Section 5, "S5 = T+4+B").

> **Remark** **Reading the diagram**
>
> Row 0 is `kCore` alone (K); row 1 adds one differentiator; row 2 adds two; row 3 (S5, D45) adds three. Every edge points from a smaller tag set to a `Finset.insert`/∪-larger one. The two into-S5 edges (from S4 and TB) are the only edges terminating at row 3 from a single-parent T/4/B combination; D45 gathers edges from all three of its two-differentiator predecessors (K45, D4, D5).

3.3 Disambiguation: this is not `Cslib/Logics/Modal/Cube.lean`

CSLib has a second, pre-existing artifact also called "the modal cube": `Cube.lean` defines each classical system as a **semantic** frame-class — a `Set (Model World Atom)` characterized by a frame property, e.g.

```
def T World Atom := logic {m : Model World Atom | Std.Refl
m.r}
```

`Cube.lean` was authored for a different purpose and predates the schema-union combinator. It must not be conflated with the syntactic tag-set cube of this section:

Artifact	File	Kind	Each system is a…
Semantic frame-class cube	`Cube.lean`	pre-existing, frame-based	`Set (Model World Atom)`
Syntactic tag-set cube	`AxiomSubsumption.lean` + `SchemaTags.lean`	new, proof-theoretic	`Finset ModalSchemaTag`

Table 3: The two independent "modal cube" artifacts living side by side. This document is about the syntactic tag-set cube only.

4 `unionSound`: The Syntax/Semantics Hinge

Durable anchors: `unionSound`, `FrameValidatesTag` (`Metalogic/SchemaSoundness.lean`); `Satisfies.modalT_axiom`, `Satisfies.modalFour_axiom`, `Satisfies.modalB_axiom`, `Satisfies.modalD_axiom`, `Satisfies.modalFive_axiom` (`Metalogic/FrameCorrespondence.lean`).

Subsumption is the syntactic collapse (Section 3); soundness is the semantic one. Just as the 24 subsumption edges collapse to one generic lemma, the 15 systems' soundness proofs collapse to one master lemma,

`unionSound`, built on a small reusable library of frame-condition-to-validity correspondence facts.

4.1 The five frame-correspondence lemmas

Each of the five modal-strength differentiator axioms is valid on any model whose accessibility relation satisfies the matching frame condition (`FrameCorrespondence.lean`), replacing five near-identical `Satisfies.modal*_axiom` blocks with one table:

Tag	Axiom schema	Frame condition	Lemma (`Satisfies.…`)
`modalT`	$\Box\phi \to \phi$	$\forall w, m.r(w,w)$ (reflexive)	`modalT_axiom`
`modalFour`	$\Box\phi \to \Box\Box\phi$	$\forall w_1 w_2 w_3, m.r(w_1,w_2) \to m.r(w_2,w_3) \to m.r(w_1,w_3)$ (transitive)	`modalFour_axiom`
`modalB`	$\phi \to \Box\Diamond\phi$[1]	$\forall w_1 w_2, m.r(w_1,w_2) \to m.r(w_2,w_1)$ (symmetric)	`modalB_axiom`
`modalD`	$\Box\phi \to \Diamond\phi$	`Relation.Serial` $m.r$ (serial)	`modalD_axiom`
`modalFive`	$\Diamond\phi \to \Box\Diamond\phi$	$\forall w_1 w_2 w_3, m.r(w_1,w_2) \to m.r(w_1,w_3) \to m.r(w_2,w_3)$ (Euclidean)	`modalFive_axiom`

Table 4: The five frame-correspondence lemmas, replacing five near-identical `Satisfies.modal*_axiom` Lean blocks. Each lemma's docstring records its provenance against Blackburn, de Rijke, and Venema, *Modal Logic* (Cambridge, 2001), Chapter 4, Definition 4.9 and Table 4.1.

4.2 `FrameValidatesTag` and `unionSound`

`FrameValidatesTag` packages the semantic obligation uniformly over all 18 tags: `True` for the 13 frame-unconditional tags, and exactly the frame-condition hypothesis for the 5 differentiators — keeping the interface `unionSound` consumes uniform, so the 13 trivial obligations discharge by `trivial` rather than requiring a type-level conditional split:

```
def FrameValidatesTag {World} (m : Model World Atom) :
ModalSchemaTag → Prop
  | .modalT => ∀ w, m.r w w
  | .modalD => Relation.Serial m.r
  | .modalB => ∀ w₁ w₂, m.r w₁ w₂ → m.r w₂ w₁
  | .modalFour => ∀ w₁ w₂ w₃, m.r w₁ w₂ → m.r w₂ w₃ → m.r
w₁ w₃
```

[1]Confirmed against live source (`Foundations/Logic/Axioms.lean:163-165`): `Axioms.AxiomB φ := φ → □(□(φ → ⊥) → ⊥)`. This is the textbook shape `φ → □◇φ` with $\Diamond$ classically encoded as $\neg\Box\neg$ (no primitive `HasDia` in this signature).

```
  | .modalFive => ∀ w₁ w₂ w₃, m.r w₁ w₂ → m.r w₁ w₃ → m.r
w₂ w₃
  | .implyK | .implyS | .efq | .peirce | .modalK
  | .andI | .andE1 | .andE2 | .orI1 | .orI2 | .orE
  | .diaDualityFwd | .diaDualityBack => True
```

`unionSound` is the master combinator — the hinge itself:

```
theorem unionSound {World : Type*} (S : Finset
ModalSchemaTag) (m : Model World Atom)
    (hfc : ∀ t ∈ S, FrameValidatesTag m t) {φ : Proposition
Atom} (h : SchemaUnion S φ)
    (w : World) : Satisfies m w φ
```

Its proof is a single `cases t with` over all 18 tags. The 13 frame-unconditional cases reuse the existing validity atoms in `Metalogic/Soundness.lean` (`Satisfies.implyK_axiom`, …, `Satisfies.diaDualityBack_axiom`) read-only, and the 5 differentiator cases each delegate directly to the corresponding `FrameCorrespondence` lemma above (e.g. `exact Satisfies.modalT_axiom m hval w φ'`) rather than re-deriving the frame argument inline.

> **Remark** **The hinge, plainly**
>
> The frame-correspondence library (semantic side) and the schema-union combinator (syntactic side) are two halves of one abstraction, and `unionSound` is the hinge between them — every per-system soundness proof specializes this one lemma instead of re-deriving a per-system case-split.

Durable anchors for this section: `Metalogic/FrameCorrespondence.lean` (the five `Satisfies.modal*_axiom` lemmas), `Metalogic/SchemaSoundness.lean` (`FrameValidatesTag`, `unionSound`), `Metalogic/Soundness.lean` (the 13 frame-unconditional validity atoms `unionSound` reuses).

5 The `HasAxiom*` Insulation Layer

Durable anchor: `Cslib/Foundations/Logic/ProofSystem.lean`.

Everything in Sections 1–3 (Section 2, Section 3, Section 4) lives at the *axiom-predicate* layer: the definition of which `Proposition` instances count as axioms for a given system. A separate, higher layer — `Foundations/Logic/ProofSystem.lean` — defines one `HasAxiom*` typeclass per schema (`HasAxiomImplyK`, `HasAxiomImplyS`, `HasAxiomEFQ`, `HasAxiomPeirce`, `HasAxiomK`, `HasAxiomT`, `HasAxiom4`, `HasAxiomB`, `HasAxiom5`, `HasAxiomD`, the and/or family, and `HasAxiomDiaDualityFwd/Back`, plus

non-modal and temporal classes outside this document's scope). A representative signature:

```
class HasAxiomT where
  T {φ : F} : InferenceSystem.DerivableIn S (Axioms.AxiomT
φ)
```

Crucially, `HasAxiomT` asserts *derivability of the axiom instance* — it says nothing about *how the underlying* `<Sys>Axiom` *predicate is constructed*. These typeclasses are bundled via `extends` into a hierarchy of Hilbert-system classes:

1. `MinimalHilbert` extends into `IntuitionisticHilbert`,
2. which extends into `ClassicalHilbert`,
3. which extends into `ModalHilbert`.

`ModalHilbert` is then extended by one class per modal-strength differentiator or combination, graph-isomorphic to the Section 2 cube (Figure 1) — each `Modal{Sys}Hilbert` class plays the role of the corresponding node:

System	Hilbert class	System	Hilbert class
K	`ModalHilbert`	TB	`ModalTBHilbert`
T	`ModalTHilbert`	KB5	`ModalKB5Hilbert`
D	`ModalDHilbert`	D4	`ModalD4Hilbert`
KB	`ModalBHilbert`	D5	`ModalD5Hilbert`
K4	`ModalK4Hilbert`	D45	`ModalD45Hilbert`
K5	`ModalK5Hilbert`	DB	`ModalDBHilbert`
S4	`ModalS4Hilbert`	S5	`ModalS5Hilbert`
K45	`ModalK45Hilbert`		

Table 5: The `extends` hierarchy's node names, in the same 15-node/24-edge shape as Figure 1: `ModalS4Hilbert extends ModalTHilbert`, `ModalK45Hilbert extends ModalK4Hilbert`, `ModalTBHilbert`/`ModalKB5Hilbert`/ `ModalD4Hilbert`/`ModalD5Hilbert`/ `ModalDBHilbert` each extend their respective single-differentiator ancestor, `ModalD45Hilbert extends ModalD4Hilbert`, up to `ModalS5Hilbert extends ModalS4Hilbert`.

> **Remark** **Same shape as the cube**
>
> This is not a coincidence: the `HasAxiom*` layer is representation-agnostic (it only ever asks for a `DerivableIn` proof), so the class hierarchy tracks the same $\subseteq$-order on differentiator sets that Section 3 already established for tag sets. Two independent layers, one shape.

Why this is "representation-agnostic insulation", not merely asserted but verified in the landed code: `Instances/S5.lean` discharges `HasAxiomT` for `Modal.HilbertS5` by constructing a `DerivationTree` witness,

```
instance : HasAxiomT Modal.HilbertS5 (F :=
Modal.Proposition Atom) where
  T := ⟨Modal.DerivationTree.ax [] _ (⟨.modalT, by decide,
_, rfl⟩)⟩
```

i.e.
a tag-membership proof (`.modalT ∈ s5Tags`, discharged by `decide`) plus a `.Holds` witness. `HasAxiomT` itself never inspects whether the underlying `<Sys>Axiom` predicate is a bespoke `inductive` or a `SchemaUnion` combinator — it only ever requires a `DerivableIn` proof. This is the load-bearing property that made the `SchemaUnion` refactor additive with zero blast radius at every downstream consumer: the `Systems/*/Completeness.lean` and `Systems/*/ConservativeExtension.lean` layers route exclusively through this `HasAxiom*`/bundled-class layer, so replacing 14 inductives with `SchemaUnion` abbreviations left them untouched.

Durable anchors for this section: `Foundations/Logic/ProofSystem.lean` (`HasAxiomT`, the full `HasAxiom*` family, `MinimalHilbert` … `ModalS5Hilbert`), `Instances/S5.lean` (the `HasAxiomT` instance witness).

6 S5 = T+4+B: The Deliberately Omitted KB5 → S5 Edge

Durable anchors: `s5Tags`, `kb5Tags` (`SchemaTags.lean`), the omission note in `AxiomSubsumption.lean`.

S5′s tag set is exactly `kCore` plus three differentiators; KB5′s is `kCore` plus two **different** differentiators:

System	Tag set
S5	`kCore` ∪ {modalT, modalFour, modalB} — **no** modalFive
KB5	`kCore` ∪ {modalB, modalFive} — **neither** modalT **nor** modalFour

Table 6: `s5Tags` and `kb5Tags` side by side (`SchemaTags.lean`).

Because `modalFive ∈ kb5Tags` and `modalFive ∉ s5Tags`, `kb5Tags ⊆ s5Tags` is false: no `Finset.subset` proof exists, and `decide` on that proposition reports `False`. Consequently `SchemaUnion.subsumption` cannot produce a `KB5Axiom_implies_ModalAxiom` lemma, and none exists in `AxiomSubsumption.lean` — visible directly in Figure 1 as the missing edge into S5 from KB5. This is not an oversight: it is a mechanical, decidable consequence of the two tag-set definitions.

> **Remark A worked example of the design's self-documenting property**
>
> Under the old per-edge hand-lemma design, a missing cube edge required a separate written justification. Under the tag-set design, a missing edge is immediately explained by inspecting two `Finset` literals — the gap is readable straight from the definitions, and any attempt to manufacture the missing edge fails at `decide` rather than at proof-search.

Durable anchors for this section: `SchemaTags.lean` (`s5Tags`, `kb5Tags`), `Metalogic/InterSystem/AxiomSubsumption.lean` (the KB5 → S5 omission note).

7 Design Rationale: Representation A vs. Representation B

This design was one of two candidate representations considered for replacing the 14 bespoke per-system axiom inductives (plus S5′s pre-existing `ModalAxiom`) with a single reusable abstraction.

- **Representation A (chosen)** — the architecture of Sections 1–5: a schema-tag `def` (`ModalSchemaTag`), an existential `.Holds` meaning function, and a `SchemaUnion (S : Finset ModalSchemaTag)` combinator parametrized by concrete `Finset` literals per system.
- **Representation B (rejected, never implemented in this codebase)** — keep the per-system `inductive <Sys>Axiom` shape, but generate each one via a macro/elaborator over a tag list (e.g. a hypothetical `derive_modal_axiom KAxiom [implyK, implyS, efq, peirce, modalK, andCore, orCore, diaDuality]`), so constructor names and elimination form (`cases h_ax with | implyK …`) would be preserved verbatim at every call site.

Representation A (chosen)	Representation B (rejected)
Subsumption: 24 hand-written lemmas → one generic `SchemaUnion.subsumption` + `decide`-able `Finset.subset` facts.	Elimination form unchanged at every downstream `cases`/`match` site — near-zero migration blast radius.
Soundness: 15 per-system case-splits → one `unionSound` application + an 18-entry validity table.	Delivers the literal DRY goal (no re-listing at the source) without touching call sites at all.
The 13-line propositional/ K/and-or/diamond-duality core lives once, in `kCore`, instead of	Does **not** deliver subsumption-as-⊆ or soundness-as-per-tag-table: cross-inductive subsump-

Representation A (chosen)	Representation B (rejected)
being re-listed per system. Net line-negative in the landed diff.	tion stays an O(edges)-sized proof set even if macro-generated, since bespoke inductive constructors still differ system to system.
Cost: the elimination form changes at every downstream destructuring site (`cases h_ax with | implyK …` becomes `obtain ⟨t, ht, hφ⟩ := h_ax; fin_cases t <;> …`) — substantially tamed by the `SchemaUnion.{empty,insert,union}_iff @[simp]` elimination API (Section 2).	Adds metaprogramming maintenance burden; less transparent/ auditable to a reviewer than a plain `def`.

Table 7: Representation A vs. Representation B, replacing the source's prose trade-off discussion.

> **Remark** **Why Representation A was chosen for long-term foundations**
>
> The goal was durable architecture, not merely a DRY-er restatement of the same 24+15 previously-independent facts. Representation A is the only one of the two that makes the modal cube's algebraic structure — a lattice of finite tag sets under $\subseteq$ — directly visible and machine-checkable in the type theory itself, rather than hiding the same bespoke facts behind macro-generated syntax. Representation B optimizes for short-term migration safety; Representation A optimizes for the property this whole document exists to describe: that subsumption and soundness are *literally* computations on `Finset ModalSchemaTag`, not just refactored restatements of previously-independent facts.

8 Scope Boundary: A Future Instance, Not a Fork

Durable anchor: the "Design Invariants" section of `SchemaUnion.lean`'s module docstring.

`ModalSchemaTag` and `SchemaUnion` were built for the 15 *classical* normal modal systems only. The module docstring states the scope boundary explicitly as a design invariant:

> `ModalSchemaTag` / `SchemaUnion` stay free of classical-only assumptions: the intuitionistic/minimal axiom families are a possible future instance of this same abstraction, not generalized to cover them here (out of scope for this task).

CSLib already has intuitionistic and minimal modal axiom families that exist today and were deliberately kept out of this design's scope:

Family	**File**
`IKModalAxiom`, `IS5ModalAxiom`	`Metalogic/Intuitionistic/{IK,IS5}.lean`
`CKModalAxiom`	`Metalogic/Constructive/CK.lean`
`MKModalAxiom`, `MTModalAxiom`	`Metalogic/Minimal/{MK,MT}.lean`

Table 8: Existing non-classical axiom families, out of this document's scope.

These families construct witnesses *into* the classical `KAxiom`/`ModalAxiom` predicates — cross-family coupling lives in `Metalogic/InterSystem/IntToClassical.lean` — so their existing call sites had to keep type-checking unchanged through the refactor described in this document. Nothing about *their own* internal representation was touched or generalized.

Remark **Future instance, not a fork**

If intuitionistic/minimal schema-union support is ever wanted, the correct move is to extend or parametrize the existing `ModalSchemaTag`/`SchemaUnion` shape — for instance by widening the tag alphabet or adding a parameter distinguishing classical from constructive instances of a tag's meaning — not to build an independent, parallel `IntSchemaTag`/`IntSchemaUnion` pair from scratch. Keeping `ModalSchemaTag`/`SchemaUnion` free of classical-only assumptions today is precisely what keeps that future extension a natural generalization of this same architecture, rather than a second, incompatible cube living alongside it.

Durable anchors for this section: `SchemaUnion.lean` (Design Invariants docstring), `Metalogic/Intuitionistic/{IK,IS5}.lean`, `Metalogic/Constructive/CK.lean`, `Metalogic/Minimal/{MK,MT}.lean`, `Metalogic/InterSystem/IntToClassical.lean`.

9 Appendix: Source Anchors

Every Lean anchor cited in this document, for a reader who wants to jump straight to source. Paths are relative to `Cslib/Logics/Modal/`, except the `HasAxiom*`/Hilbert-class family and the `AxiomB` confirmation, which live under `Cslib/Foundations/Logic/`. Line numbers were verified against live, CI-green source during authoring of this document.

File : lines	Declaration (section)
`ProofSystem/SchemaUnion.lean:67`	`ModalSchemaTag` (§1, 18-tag alphabet)
`ProofSystem/SchemaUnion.lean:115`	`ModalSchemaTag.Holds` (§1, schema meaning)
`ProofSystem/SchemaUnion.lean:146`	`SchemaUnion` (§1, the combinator)
`ProofSystem/SchemaUnion.lean:155`	`SchemaUnion.subsumption` (§2, generic subsumption)
`ProofSystem/SchemaUnion.lean:171,180,195`	`SchemaUnion.{empty,insert,union}_iff` `@[simp]` API (§1, `@[simp]` API)
`ProofSystem/SchemaUnion.lean:39-47`	module docstring, Design Invariants (§7, scope boundary)
`Metalogic/DerivationTree.lean:69`	`ModalAxiom := SchemaUnion s5Tags` (§1, S5 folded in)
`ProofSystem/SchemaTags.lean:57`	`kCore` (§2, the 13-tag shared core)
`ProofSystem/SchemaTags.lean:92`	`s5Tags` (§2, §5, S5′s tag set)
`ProofSystem/SchemaTags.lean:99`	`kb5Tags` (§2, §5, KB5′s tag set)
`Metalogic/InterSystem/AxiomSubsumption.lean:65`	KB5 → S5 omission note (§5)
`Logics/Modal/Cube.lean`	semantic frame-class cube (§2, disambiguation)
`Metalogic/FrameCorrespondence.lean:56,64,73,81,91`	the five `Satisfies.modal*_axiom` lemmas (§3)
`Metalogic/SchemaSoundness.lean:70`	`FrameValidatesTag` (§3)
`Metalogic/SchemaSoundness.lean:88`	`unionSound` (§3, the hinge)
`Foundations/Logic/Axioms.lean:163-165`	`Axioms.AxiomB` (§3, confirmed expansion)
`Foundations/Logic/ProofSystem.lean:178`	`HasAxiomT` (§4, representative typeclass)
`Foundations/Logic/ProofSystem.lean:342,349,355,361`	`MinimalHilbert` … `ModalHilbert` (§4, base chain)
`Foundations/Logic/ProofSystem.lean:386`	`ModalS5Hilbert` (§4, top of the composite chain)
`ProofSystem/Instances/S5.lean:81`	`HasAxiomT Modal.HilbertS5` instance (§4, witness)

File : lines	Declaration (section)
`Metalogic/Intuitionistic/IK.lean:75`	`IKModalAxiom` (§7, out of scope)
`Metalogic/Constructive/CK.lean:104`	`CKModalAxiom` (§7, out of scope)
`Metalogic/Minimal/MK.lean`, `MT.lean`	`MKModalAxiom`, `MTModalAxiom` (§7, out of scope)

Table 9: Verified Lean source anchors cited in this document.